

ROS

Ricardo Victoria

octubre 2027

1. Espacio de trabajo y Paquetes

1.1. Estructura General

Cuando utilizamos ROS 2, debemos organizar nuestras carpetas y archivos de una forma muy específica. De manera muy general, la organización va así:

espacio de trabajo>paquetes>nodos de ros.

Un espacio de trabajo es una carpeta que contiene todas las carpetas y archivos de nuestro proyecto. Un paquete es una carpeta que contiene los nodos que programemos. Un nodo es un archivo de código fuente, usualmente un objeto.

Más detalladamente, la estructura de un espacio de trabajo de ros se ve así:

```

ros2_ws/ .....Espacio de trabajo (Workspace)
├── src/ .....Directorio raíz del código fuente
│   ├── mi_paquete_cpp/ .....Paquete de C++ (ament_cmake)
│   │   ├── CMakeLists.txt .....Instrucciones de compilación de CMake
│   │   ├── package.xml .....Declaración de dependencias y metadatos
│   │   ├── include/ .....Archivos de cabecera (.hpp)
│   │   └── src/ .....Código fuente ejecutable
│   │       └── nodo_cpp.cpp .....Nodo escrito en C++
│   ├── mi_paquete_py/ .....Paquete de Python (ament_python)
│   │   ├── package.xml .....Declaración de dependencias y metadatos
│   │   ├── setup.py .....Configuración de instalación de Python
│   │   ├── setup.cfg .....Definición de ejecutables/puntos de entrada
│   │   └── mi_paquete_py/ .....Módulo principal de Python
│   │       └── nodo_py.py .....Nodo escrito en Python
├── build/ .....Archivos intermedios (generado por colcon)
├── install/ .....Puntos de entrada y binarios (generado por colcon)
└── log/ .....Registros de depuración (generado por colcon)

```

En el diagrama anterior, se muestra la estructura de un espacio de trabajo de ROS, con dos paquetes, uno de C++ y otro de Python.

1.2. Creación de un Espacio de Trabajo

Vamos a crear un espacio de trabajo en la terminal de Linux. Primero, creamos una carpeta que será el espacio de trabajo, a esta carpeta la nombraremos `taller_ws` (nótese que `ws` viene de `workspace` o espacio de trabajo). Entonces creamos la carpeta con `mkdir taller_ws`.

```
usuario@equipo:~/ROS$ mkdir taller_ws
```

Luego, dentro del espacio de trabajo crearemos una carpeta llamada `src` (esa carpeta por convención se llama así).

```
usuario@equipo:~/ROS$ cd taller_ws
usuario@equipo:~/ROS/taller_ws$ mkdir src
```

Dentro de `src`, vamos a hacer un paquete de C++ llamado `viz_package_cpp`. Queremos que este paquete contenga:

- un archivo `CMakeLists.txt`
- una carpeta `include/`
- una licencia Apache 2.0
- un archivo `package.xml`
- una carpeta con un nodo `src/viz_node.cpp`

Lo anterior lo podemos hacer con el siguiente comando:

```
ros2 pkg create --build-type ament_cmake --node-name \
viz_node viz_package_cpp --license Apache-2.0
```

```
usuario@equipo:~/ROS/taller_ws$ cd src
```

```
usuario@equipo:~/ROS/taller_ws/src$ ros2 pkg create --build-type ament_cmake \
--node-name viz_node viz_package_cpp --license Apache-2.0
```

```

ricky@ThinkPad-T14-Gen-2i:~$ mkdir ROS
ricky@ThinkPad-T14-Gen-2i:~$ cd ROS
ricky@ThinkPad-T14-Gen-2i:~/ROS$ mkdir taller_ws
ricky@ThinkPad-T14-Gen-2i:~/ROS$ cd taller_ws/
ricky@ThinkPad-T14-Gen-2i:~/ROS/taller_ws$ mkdir src
ricky@ThinkPad-T14-Gen-2i:~/ROS/taller_ws$ cd src/
ricky@ThinkPad-T14-Gen-2i:~/ROS/taller_ws/src$ ros2 pkg create --build-type ament_cmake --node-name \
viz_node viz_package_cpp --license Apache-2.0
going to create a new package
package name: viz_package_cpp
destination directory: /home/ricky/ROS/taller_ws/src
package format: 3
version: 0.0.0
description: TODO: Package description
maintainer: ['ricky <ricardo.max.vic.mor@gmail.com>']
licenses: ['Apache-2.0']
build type: ament_cmake
dependencies: []
node_name: viz_node
creating folder ./viz_package_cpp
creating ./viz_package_cpp/package.xml
creating source and include folder
creating folder ./viz_package_cpp/src
creating folder ./viz_package_cpp/include/viz_package_cpp
creating ./viz_package_cpp/CMakeLists.txt
creating ./viz_package_cpp/src/viz_node.cpp
ricky@ThinkPad-T14-Gen-2i:~/ROS/taller_ws/src$

```

Figura 1: Creación de Espacio de Trabajo, y Paquete con Nodo.

A continuación se muestra cómo quedó la estructura del paquete.

```

usuario@equipo:~/ROS/taller_ws/src$ tree

```

```

.
├── viz_package_cpp
│   ├── CMakeLists.txt
│   ├── include
│   │   └── viz_package_cpp
│   ├── LICENSE
│   ├── package.xml
│   └── src
│       └── viz_node.cpp

```

5 directories, 4 files

```
ricky@ThinkPad-T14-Gen-2i:~/ROS/taller_ws/src$ tree
.
├── viz_package_cpp
│   ├── CMakeLists.txt
│   ├── include
│   │   └── viz_package_cpp
│   ├── LICENSE
│   ├── package.xml
│   └── src
│       └── viz_node.cpp
5 directories, 4 files
ricky@ThinkPad-T14-Gen-2i:~/ROS/taller_ws/src$
```

Figura 2: Estructura del Paquete.

[en otras iteraciones, explicaré el contenido de los archivos anteriores].

1.3. Compilar y Ejecutar

Una vez que tenemos la estructura que definimos anteriormente. Procedemos a compilar el espacio de trabajo, y ejecutar el nodo que creamos. El nodo que creamos anteriormente sólo devuelve un hola mundo.

Para compilar, usamos `colcon`, la herramienta de compilación por línea de comandos oficial de ROS 2. Su función es orquestar la construcción de múltiples paquetes dentro de un espacio de trabajo.

Podemos compilar e instalar todo el espacio de trabajo con:

```
colcon build
```

o podemos compilar e instalar un paquete específico con:

```
colcon build --packages-select <nombre del paquete>
```

```
ricky@ThinkPad-T14-Gen-2i:~/ROS/taller_ws/src$ cd ..
ricky@ThinkPad-T14-Gen-2i:~/ROS/taller_ws$ colcon build
Starting >>> viz_package_cpp
Finished <<< viz_package_cpp [1.63s]

Summary: 1 package finished [1.80s]
ricky@ThinkPad-T14-Gen-2i:~/ROS/taller_ws$ █
```

Figura 3: Compilación.

Luego, para ejecutar un nodo de un paquete, abrimos una nueva terminal (no se debe ejecutar en la misma terminal donde se compiló, porque puede fallar la ejecución), y en la nueva terminal activamos el overlay con:

```
source install/local_setup.bash
```

y ejecutamos el nodo con:

```
ros2 run <nombre del paquete> <nombre del nodo>
```

```
ricky@ThinkPad-T14-Gen-2i:~/ROS/taller_ws$ source install/local_setup.bash
ricky@ThinkPad-T14-Gen-2i:~/ROS/taller_ws$ ros2 run viz_package_cpp viz_node
hello world viz_package_cpp package
ricky@ThinkPad-T14-Gen-2i:~/ROS/taller_ws$ █
```

Figura 4: Ejecución del Nodo.

2. Archivos de Lanzamiento

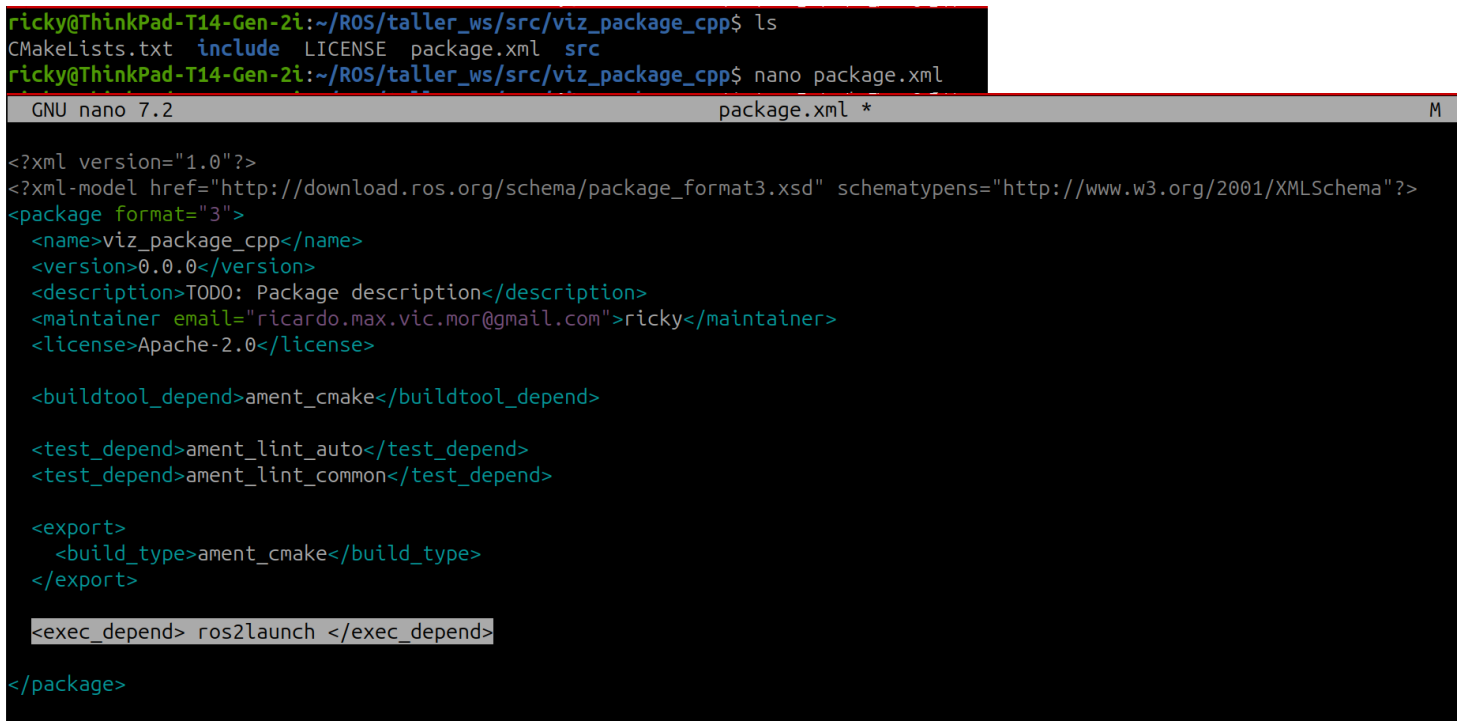
Anteriormente vimos cómo ejecutar un nodo. Pero en un proyecto grande de ROS, necesitaremos lanzar muchos más nodos. Necesitamos una forma de ejecutar muchos nodos con un sólo comando. Para esos sirven los archivos de lanzamiento o launch files.

Un launch file permite realizar varias configuraciones para el ambiente y ejecutar multiples nodos.

Para poder usar launch files, realizar los siguientes pasos:

1. Dentro del archivo `package.xml` del paquete, agregar la siguiente línea dentro de las etiquetas `<package>`:

```
<exec_depend> ros2launch </exec_depend>
```



```
ricky@ThinkPad-T14-Gen-2i:~/ROS/taller_ws/src/viz_package_cpp$ ls
CMakeLists.txt  include  LICENSE  package.xml  src
ricky@ThinkPad-T14-Gen-2i:~/ROS/taller_ws/src/viz_package_cpp$ nano package.xml
GNU nano 7.2 package.xml *
<?xml version="1.0"?>
<?xml-model href="http://download.ros.org/schema/package_format3.xsd" schematypens="http://www.w3.org/2001/XMLSchema"?>
<package format="3">
  <name>viz_package_cpp</name>
  <version>0.0.0</version>
  <description>TODO: Package description</description>
  <maintainer email="ricardo.max.vic.mor@gmail.com">ricky</maintainer>
  <license>Apache-2.0</license>

  <buildtool_depend>ament_cmake</buildtool_depend>

  <test_depend>ament_lint_auto</test_depend>
  <test_depend>ament_lint_common</test_depend>

  <export>
    <build_type>ament_cmake</build_type>
  </export>

  <exec_depend> ros2launch </exec_depend>
</package>
```

Figura 5: Edición de package.xml para habilitar launch files.

2. Crear una carpeta `launch` dentro del paquete.

```
ricky@ThinkPad-T14-Gen-2i:~/ROS/taller_ws/src/viz_package_cpp$ mkdir launch
ricky@ThinkPad-T14-Gen-2i:~/ROS/taller_ws/src/viz_package_cpp$ ls
CMakeLists.txt  include  launch  LICENSE  package.xml  src
ricky@ThinkPad-T14-Gen-2i:~/ROS/taller_ws/src/viz_package_cpp$
```

Figura 6: Creación carpeta launch.

3. Dentro de la carpeta `launch`, creamos un archivo de lanzamiento `viz_launch.py` con el contenido que se ve en la imagen siguiente.

```
ricky@ThinkPad-T14-Gen-2i:~/ROS/taller_ws/src/viz_package_cpp/launch$ echo "
from launch import LaunchDescription
from launch_ros.actions import Node

def generate_launch_description():
    return LaunchDescription(
        [
            Node(
                package="viz_package_cpp",
                namespace="paquito1",
                executable="viz_node",
                name="viz",
                output="screen",
            )
        ]
    )" >> viz_launch.py
ricky@ThinkPad-T14-Gen-2i:~/ROS/taller_ws/src/viz_package_cpp/launch$
```

Figura 7: Creación carpeta launch.

4. Volvemos a compilar.

Con los pasos anteriores, ya podemos hacer uso de nuestro archivo de lanzamiento `viz_launch.py`, el cual lanza el nodo `viz_node`. Para lanzar el launch file nos ubicamos en la carpeta `launch` del paquete, y hacemos:

```
ros2 launch <nombre archivo lanzamiento>
```

```
ricky@ThinkPad-T14-Gen-2i:~/ROS/taller_ws$ colcon build
Starting >>> viz_package_cpp
Finished <<< viz_package_cpp [0.14s]

Summary: 1 package finished [0.29s]
ricky@ThinkPad-T14-Gen-2i:~/ROS/taller_ws$

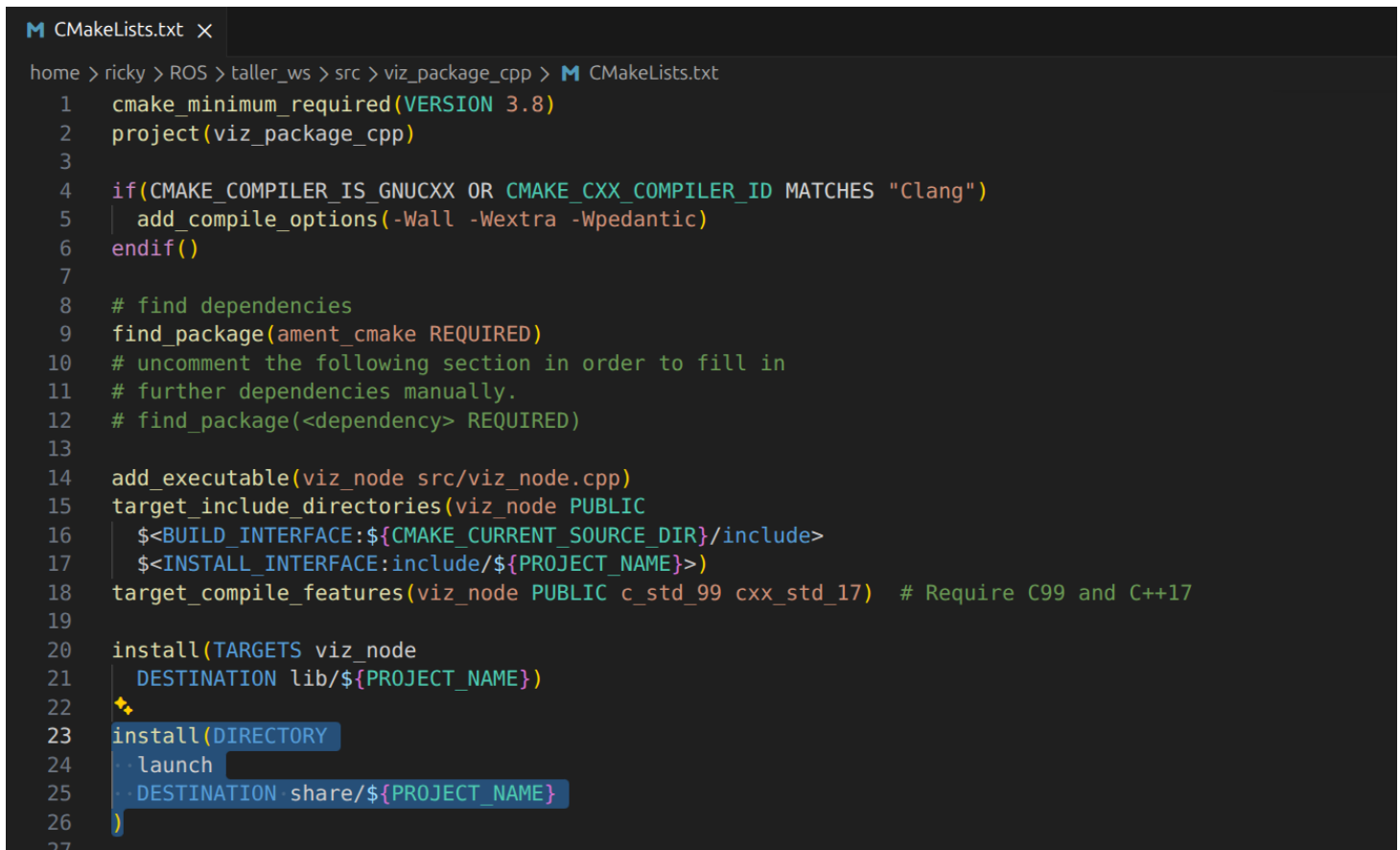
ricky@ThinkPad-T14-Gen-2i:~/ROS/taller_ws/src/viz_package_cpp/launch$ ros2 launch viz_launch.py
[INFO] [launch]: All log files can be found below /home/ricky/.ros/log/2026-10-04-18-22-40-207767-ThinkPad-T14-Gen-2i-33502
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [viz_node-1]: process started with pid [33505]
[viz_node-1] hello world viz_package_cpp package
[INFO] [viz_node-1]: process has finished cleanly [pid 33505]
ricky@ThinkPad-T14-Gen-2i:~/ROS/taller_ws/src/viz_package_cpp/launch$
```

Figura 8: Lanzando nuestro primer launch file.

Ahora, para poder lanzar el launch file sin tener que ubicarnos en la carpeta launch, hacemos lo siguiente:

1. Abrir el archivo CMakeLists.txt del paquete, y agregar lo siguiente:

```
install(DIRECTORY
  launch
  DESTINATION share/${PROJECT_NAME}
)
```



```
M CMakeLists.txt X
home > ricky > ROS > taller_ws > src > viz_package_cpp > M CMakeLists.txt
1  cmake_minimum_required(VERSION 3.8)
2  project(viz_package_cpp)
3
4  if(CMAKE_COMPILER_IS_GNUCXX OR CMAKE_CXX_COMPILER_ID MATCHES "Clang")
5    add_compile_options(-Wall -Wextra -Wpedantic)
6  endif()
7
8  # find dependencies
9  find_package(ament_cmake REQUIRED)
10 # uncomment the following section in order to fill in
11 # further dependencies manually.
12 # find_package(<dependency> REQUIRED)
13
14 add_executable(viz_node src/viz_node.cpp)
15 target_include_directories(viz_node PUBLIC
16   ${BUILD_INTERFACE:${CMAKE_CURRENT_SOURCE_DIR}/include}
17   ${INSTALL_INTERFACE:include/${PROJECT_NAME}})
18 target_compile_features(viz_node PUBLIC c_std_99 cxx_std_17) # Require C99 and C++17
19
20 install(TARGETS viz_node
21   DESTINATION lib/${PROJECT_NAME})
22
23 install(DIRECTORY
24   · launch
25   · DESTINATION share/${PROJECT_NAME}
26 )
27
```

Figura 9: Editar CMakeLists.txt para que los archivos launch se guarden en la carpeta share.

2. En una terminal y en el espacio de trabajo, compilar con `colcon build`.
3. En otra terminal y en el espacio de trabajo, activar el overlay con `source install/local_setup.bash` y lanzar el launch file con `ros2 launch viz_package_cpp viz_launch.py`

```

ricky@ThinkPad-T14-Gen-2i:~/ROS/taller_ws$ colcon build
Starting >>> viz_package_cpp
Finished <<< viz_package_cpp [0.57s]

Summary: 1 package finished [0.74s]
ricky@ThinkPad-T14-Gen-2i:~/ROS/taller_ws$

ricky@ThinkPad-T14-Gen-2i:~/ROS/taller_ws$ source install/local_setup.bash
ricky@ThinkPad-T14-Gen-2i:~/ROS/taller_ws$ ros2 launch viz_package_cpp viz_launch.py
[INFO] [launch]: All log files can be found below /home/ricky/.ros/log/2026-10-04-18-43-56-579657-ThinkPad-T14-Gen-2i-34582
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [viz_node-1]: process started with pid [34585]
[viz_node-1] hello world viz_package_cpp package
[INFO] [viz_node-1]: process has finished cleanly [pid 34585]
ricky@ThinkPad-T14-Gen-2i:~/ROS/taller_ws$ cd
ricky@ThinkPad-T14-Gen-2i:~$ ros2 launch viz_package_cpp viz_launch.py
[INFO] [launch]: All log files can be found below /home/ricky/.ros/log/2026-10-04-18-44-02-896310-ThinkPad-T14-Gen-2i-34586
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [viz_node-1]: process started with pid [34589]
[viz_node-1] hello world viz_package_cpp package
[INFO] [viz_node-1]: process has finished cleanly [pid 34589]
ricky@ThinkPad-T14-Gen-2i:~$ █

```

Figura 10: Ahora se puede lanzar el launch file en cualquier ruta.

3. Modelado de Robots

Ahora veremos cómo podemos construir y ver una visualización de un objeto en el programa rviz. Con eso, podremos construir una visualización de un robot.

Primero, creamos una carpeta llamada `urdf` dentro del paquete (justo debajo del paquete). Acá corregir, porque la imagen muestra que puse la carpeta en el espacio de trabajo y no en el paquete.

```

ricky@ThinkPad-T14-Gen-2i:~/ROS/taller_ws$ mkdir urdf
ricky@ThinkPad-T14-Gen-2i:~/ROS/taller_ws$ ls
build install log src urdf
ricky@ThinkPad-T14-Gen-2i:~/ROS/taller_ws$

```

Figura 11: Creación de carpeta urdf dentro del paquete.

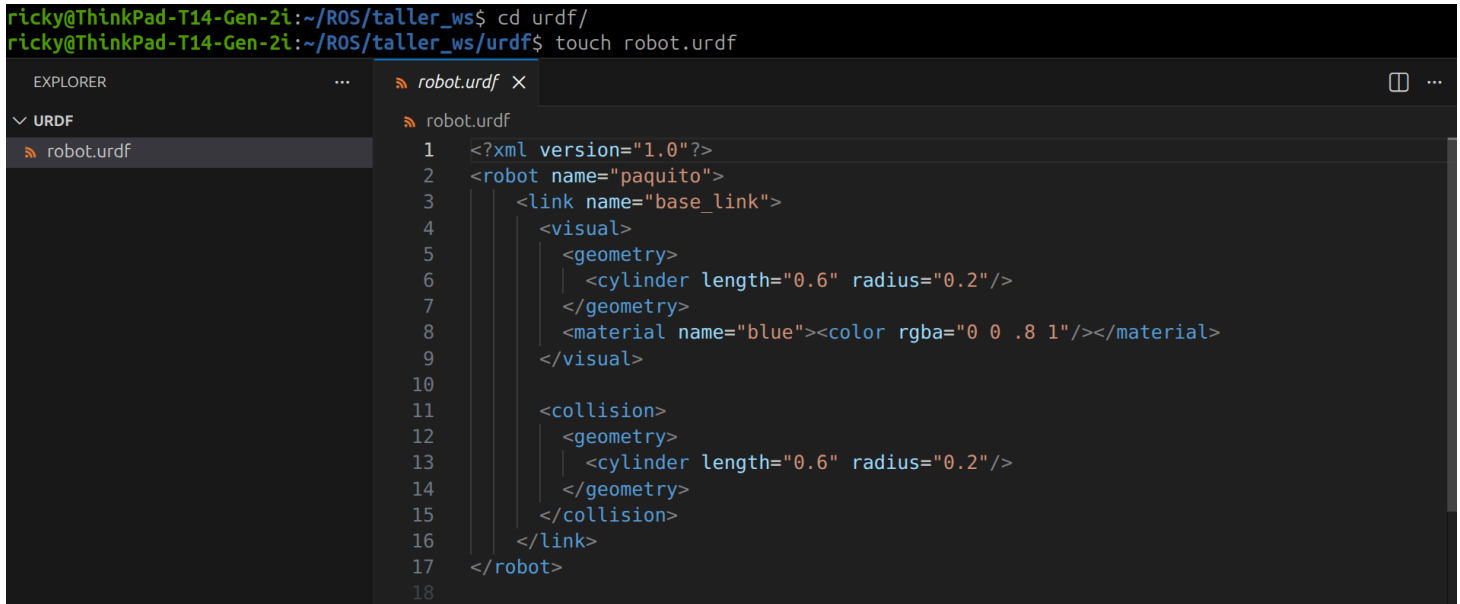
Ahora, vamos a crear un archivo `robot.urdf` dentro de la carpeta `urdf/`. `robot.urdf` contendrá lo siguiente:

```

<?xml version="1.0"?>
<robot name="paquito">
  <link name="base_link">
    <visual>
      <geometry>
        <cylinder length="0.6" radius="0.2"/>
      </geometry>
      <material name="blue"><color rgba="0 0 .8 1"/></material>
    </visual>

    <collision>
      <geometry>
        <cylinder length="0.6" radius="0.2"/>
      </geometry>
    </collision>
  </link>
</robot>

```



The screenshot shows a terminal window at the top with the following commands and output:

```

ricky@ThinkPad-T14-Gen-2i:~/ROS/taller_ws$ cd urdf/
ricky@ThinkPad-T14-Gen-2i:~/ROS/taller_ws/urdf$ touch robot.urdf

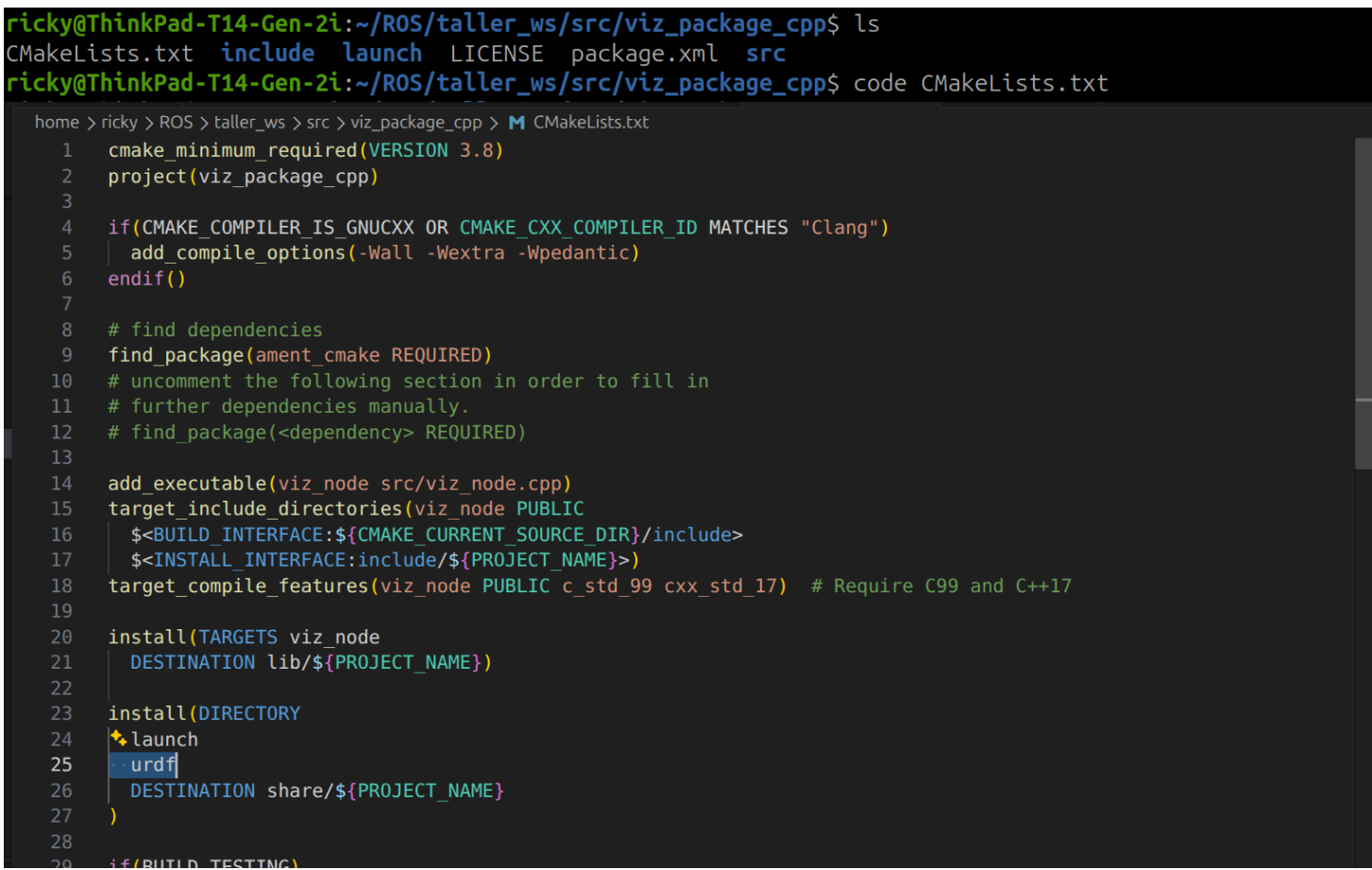
```

Below the terminal is a code editor window with a file explorer on the left. The file explorer shows a folder named 'URDF' containing a file named 'robot.urdf'. The main editor area displays the XML content of 'robot.urdf' with line numbers from 1 to 18. The XML code is identical to the one shown in the previous block.

Figura 12: Creación de archivo robot.urdf.

Luego, modificamos la línea `urdf` a la directiva `install` en el archivo `CMakeLists.txt`. La directiva `install` debe quedar como sigue:

```
install (DIRECTORY
launch
urdf # nueva línea
DESTINATION share/${PROJECT_NAME}
)
```



```
ricky@ThinkPad-T14-Gen-2i:~/ROS/taller_ws/src/viz_package_cpp$ ls
CMakeLists.txt include launch LICENSE package.xml src
ricky@ThinkPad-T14-Gen-2i:~/ROS/taller_ws/src/viz_package_cpp$ code CMakeLists.txt

home > ricky > ROS > taller_ws > src > viz_package_cpp > M CMakeLists.txt
1  cmake_minimum_required(VERSION 3.8)
2  project(viz_package_cpp)
3
4  if(CMAKE_COMPILER_IS_GNUCXX OR CMAKE_CXX_COMPILER_ID MATCHES "Clang")
5  |  add_compile_options(-Wall -Wextra -Wpedantic)
6  |  endif()
7
8  # find dependencies
9  find_package(ament_cmake REQUIRED)
10 # uncomment the following section in order to fill in
11 # further dependencies manually.
12 # find_package(<dependency> REQUIRED)
13
14 add_executable(viz_node src/viz_node.cpp)
15 target_include_directories(viz_node PUBLIC
16 |  ${<BUILD_INTERFACE:${CMAKE_CURRENT_SOURCE_DIR}/include>
17 |  ${<INSTALL_INTERFACE:include/${PROJECT_NAME}>})
18 target_compile_features(viz_node PUBLIC c_std_99 cxx_std_17) # Require C99 and C++17
19
20 install(TARGETS viz_node
21 |  DESTINATION lib/${PROJECT_NAME})
22
23 install(DIRECTORY
24 |  * launch
25 |  * urdf
26 |  DESTINATION share/${PROJECT_NAME}
27 |  )
28
29 if(BUILD_TESTING)
```

Figura 13: Agregar `urdf` a la directiva `install` del `CMakeLists.txt`.

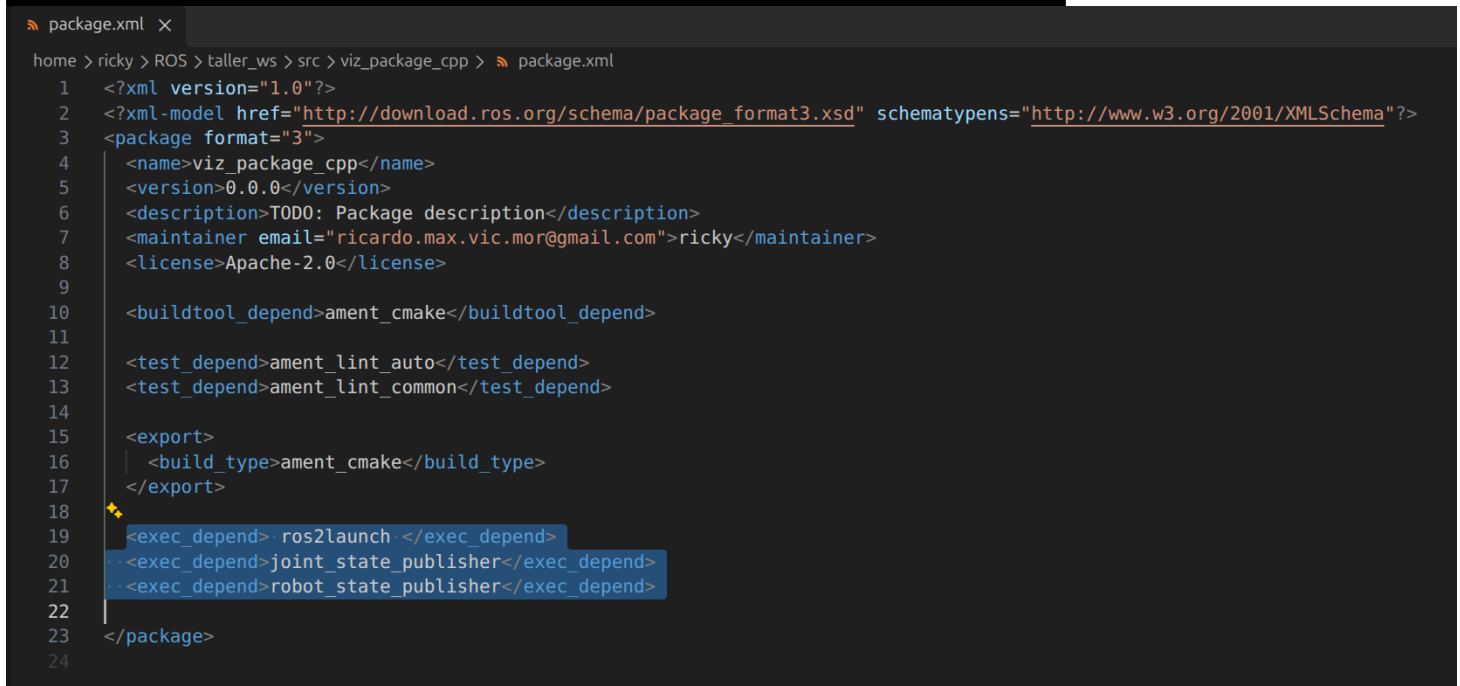
Ahora que ya tenemos la descripción de la forma del objeto a visualizar,

pasaremos a la parte del movimiento. En el archivo `package.xml` del paquete, agregamos las siguientes líneas:

```
<exec_depend>joint_state_publisher</exec_depend>
<exec_depend>robot_state_publisher</exec_depend>
```

```
ricky@ThinkPad-T14-Gen-2i:~/ROS/taller_ws$ colcon build
Starting >>> viz_package_cpp
Finished <<< viz_package_cpp [0.14s]

Summary: 1 package finished [0.29s]
ricky@ThinkPad-T14-Gen-2i:~/ROS/taller_ws$
```



```
package.xml x
home > ricky > ROS > taller_ws > src > viz_package_cpp > package.xml
1  <?xml version="1.0"?>
2  <?xml-model href="http://download.ros.org/schema/package_format3.xsd" schematypens="http://www.w3.org/2001/XMLSchema"?>
3  <package format="3">
4    <name>viz_package_cpp</name>
5    <version>0.0.0</version>
6    <description>TODO: Package description</description>
7    <maintainer email="ricardo.max.vic.mor@gmail.com">ricky</maintainer>
8    <license>Apache-2.0</license>
9
10   <buildtool_depend>ament_cmake</buildtool_depend>
11
12   <test_depend>ament_lint_auto</test_depend>
13   <test_depend>ament_lint_common</test_depend>
14
15   <export>
16     <build_type>ament_cmake</build_type>
17   </export>
18
19   <exec_depend> ros2launch </exec_depend>
20   <exec_depend>joint_state_publisher</exec_depend>
21   <exec_depend>robot_state_publisher</exec_depend>
22
23 </package>
24
```

Figura 14: Modificación `package.xml`.

Ejecutar el siguiente comando en el espacio de trabajo:

```
rosdep install -i --from-path src --rosdistro jazzy -y
```

Si pide inicializar, ejecutar los comandos siguientes antes de ejecutar el comando anterior:

```
sudo rosdep init  
rosdep update
```

```

ricky@ThinkPad-T14-Gen-2i:~/ROS/taller_ws$ rosdep install -i --from-path src --rosdistro jazzy -y
ERROR: your rosdep installation has not been initialized yet. Please run:

    sudo rosdep init
    rosdep update

ricky@ThinkPad-T14-Gen-2i:~/ROS/taller_ws$ sudo rosdep init
[sudo] password for ricky:
Wrote /etc/ros/rosdep/sources.list.d/20-default.list
Recommended: please run

    rosdep update

ricky@ThinkPad-T14-Gen-2i:~/ROS/taller_ws$ rosdep update
reading in sources list data from /etc/ros/rosdep/sources.list.d
Hit https://raw.githubusercontent.com/ros/rosdistro/master/rosdep/osx-homebrew.yaml
Hit https://raw.githubusercontent.com/ros/rosdistro/master/rosdep/base.yaml
Hit https://raw.githubusercontent.com/ros/rosdistro/master/rosdep/python.yaml
Hit https://raw.githubusercontent.com/ros/rosdistro/master/rosdep/ruby.yaml
Hit https://raw.githubusercontent.com/ros/rosdistro/master/releases/fuerte.yaml
Query rosdistro index https://raw.githubusercontent.com/ros/rosdistro/master/index-v4.yaml
Skip end-of-life distro "ardent"
Skip end-of-life distro "bouncy"
Skip end-of-life distro "crystal"
Skip end-of-life distro "dashing"
Skip end-of-life distro "eloquent"
Skip end-of-life distro "foxy"
Skip end-of-life distro "galactic"
Skip end-of-life distro "groovy"
Add distro "humble"
Skip end-of-life distro "hydro"
Skip end-of-life distro "indigo"
Skip end-of-life distro "iron"
Skip end-of-life distro "jade"
Add distro "jazzy"
Add distro "kilted"
Skip end-of-life distro "kinetic"
Skip end-of-life distro "lunar"
Add distro "lyrical"
Skip end-of-life distro "melodic"
Skip end-of-life distro "noetic"
Add distro "rolling"
updated cache in /home/ricky/.ros/rosdep/sources.cache
ricky@ThinkPad-T14-Gen-2i:~/ROS/taller_ws$ rosdep install -i --from-path src --rosdistro jazzy -y
executing command [sudo -H apt-get install -y ros-jazzy-joint-state-publisher]
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following packages were automatically installed and are no longer required:
  libfwupd2 libglib-amber-dri libglapi-mesa linux-headers-7.0.0-28-generic linux-hwe-7.0-headers-7.0.0-28 linux-hwe-7.0-tools-7.0.0-28
  linux-image-7.0.0-28-generic linux-modules-7.0.0-28-generic linux-tools-7.0.0-28-generic
Use 'sudo apt autoremove' to remove them.
The following NEW packages will be installed:
  ros-jazzy-joint-state-publisher
0 upgraded, 1 newly installed, 0 to remove and 2 not upgraded.
Need to get 19.7 kB of archives.
After this operation, 85.0 kB of additional disk space will be used.
Get:1 http://packages.ros.org/ros2/ubuntu noble/main amd64 ros-jazzy-joint-state-publisher amd64 2.4.3-1noble.20260902.054703 [19.7 kB]
Fetched 19.7 kB in 2s (11.1 kB/s)
Selecting previously unselected package ros-jazzy-joint-state-publisher.
(Reading database ... 598330 files and directories currently installed.)
Preparing to unpack .../ros-jazzy-joint-state-publisher_2.4.3-1noble.20260902.054703_amd64.deb ...
Unpacking ros-jazzy-joint-state-publisher (2.4.3-1noble.20260902.054703) ...
Setting up ros-jazzy-joint-state-publisher (2.4.3-1noble.20260902.054703) ...
#All required rosdeps installed successfully
ricky@ThinkPad-T14-Gen-2i:~/ROS/taller_ws$

```

Figura 15: Instalación con rosdep.

Ahora, hay que pedir al archivo de lanzamiento que ejecute al nodo que publicará la descripción del robot. Modificamos al archivo `viz_launch.py` para que quede de la siguiente forma.

```
import os
from ament_index_python.packages import get_package_share_directory
from launch import LaunchDescription
from launch_ros.actions import Node

def generate_launch_description():
    package_dir = get_package_share_directory('viz_package_cpp')
    path_to_urdf = os.path.join(package_dir, 'urdf', 'robot.urdf')

    with open(path_to_urdf, 'r') as f:
        robot_desc = f.read()

    return LaunchDescription([
        Node(
            package='viz_package_cpp',
            namespace='paquito1',
            executable='viz_node',
            name='viz',
            output='screen',
        ),
        Node(
            package='robot_state_publisher',
            name='robot_state_publisher',
            executable='robot_state_publisher',
            output='screen',
            parameters=[{
                'robot_description': robot_desc,
                'publish_frequency': 30.0,
            }]
        )
    ])
```

```
viz_launch.py x
home > ricky > ROS > taller_ws > src > viz_package_cpp > launch > viz_launch.py > generate_launch_description

1  import os
2  from ament_index_python.packages import get_package_share_directory
3  from launch import LaunchDescription
4  from launch_ros.actions import Node
5
6
7  def generate_launch_description():
8      package_dir = get_package_share_directory('viz_package_cpp')
9      path_to_urdf = os.path.join(package_dir, 'urdf', 'robot.urdf')
10
11      with open(path_to_urdf, 'r') as f:
12          robot_desc = f.read()
13
14      return LaunchDescription([
15          Node(
16              package='viz_package_cpp',
17              namespace='paquito1',
18              executable='viz_node',
19              name='viz',
20              output='screen',
21          ),
22          Node(
23              package='robot_state_publisher',
24              name='robot_state_publisher',
25              executable='robot_state_publisher',
26              output='screen',
27              parameters=[{
28                  'robot_description': robot_desc,
29                  'publish_frequency': 30.0,
30              }]
31          )
32      ])
```

Figura 16: Modificación al launch file para publicar la figura.

Ahora procedemos a compilar y lanzar el launchfile. Recordemos que esto se hace con los siguientes comandos y como sigue:

```
colcon build
ros2 launch viz_package_cpp viz_launch.py
```

```
ricky@ThinkPad-T14-Gen-2i: ~/ROS/taller_ws 126x28
ricky@ThinkPad-T14-Gen-2i:~/ROS/taller_ws$ colcon build
Starting >>> viz_package_cpp
Finished <<< viz_package_cpp [0.14s]

Summary: 1 package finished [0.29s]
ricky@ThinkPad-T14-Gen-2i:~/ROS/taller_ws$

ricky@ThinkPad-T14-Gen-2i:~/ROS/taller_ws 126x28
ricky@ThinkPad-T14-Gen-2i:~/ROS/taller_ws$ source install/local_setup.bash
ricky@ThinkPad-T14-Gen-2i:~/ROS/taller_ws$ ros2 launch viz_package_cpp viz_launch.py
[INFO] [launch]: All log files can be found below /home/ricky/.ros/log/2026-10-05-00-06-52-814148-ThinkPad-T14-Gen-2i-74879
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [viz_node-1]: process started with pid [74882]
[INFO] [viz_node-1]: process has finished cleanly [pid 74882]
[INFO] [robot_state_publisher-2]: process started with pid [74883]
[viz_node-1] hello world viz_package_cpp package
[robot_state_publisher-2] [INFO] [1791180412.944926757] [robot_state_publisher]: Robot initialized

ricky@ThinkPad-T14-Gen-2i:~/ROS/taller_ws 126x28
ricky@ThinkPad-T14-Gen-2i:~/ROS/taller_ws$ ros2 topic list
/joint_states
/parameter_events
/robot_description
/rosout
/tf
/tf_static
ricky@ThinkPad-T14-Gen-2i:~/ROS/taller_ws$
```

Figura 17: Compilación, lanzamiento de launch file y listado de tópicos.

Vemos de la figura anterior, en la terminal 2, que se sigue ejecutando el nodo de hola mundo, y además se ejecuta el segundo nodo que publica la descripción del robot. Podemos comprobar que se publica la descripción del robot en la terminal 3, porque se publica el tópico `/robot_description`.

4. Rviz

Continuaremos con lo anterior. Veremos cómo podemos abrir el programa de visualización rviz utilizando el launch file.

Creamos una carpeta `rviz` al lado de la carpeta `urdf`. Luego abrimos rviz en terminal con el comando `rviz2`.

`rviz2`

Dentro de rviz, en el menú superior, en file, seleccionar Save Config As, como se muestra en la siguiente imagen:

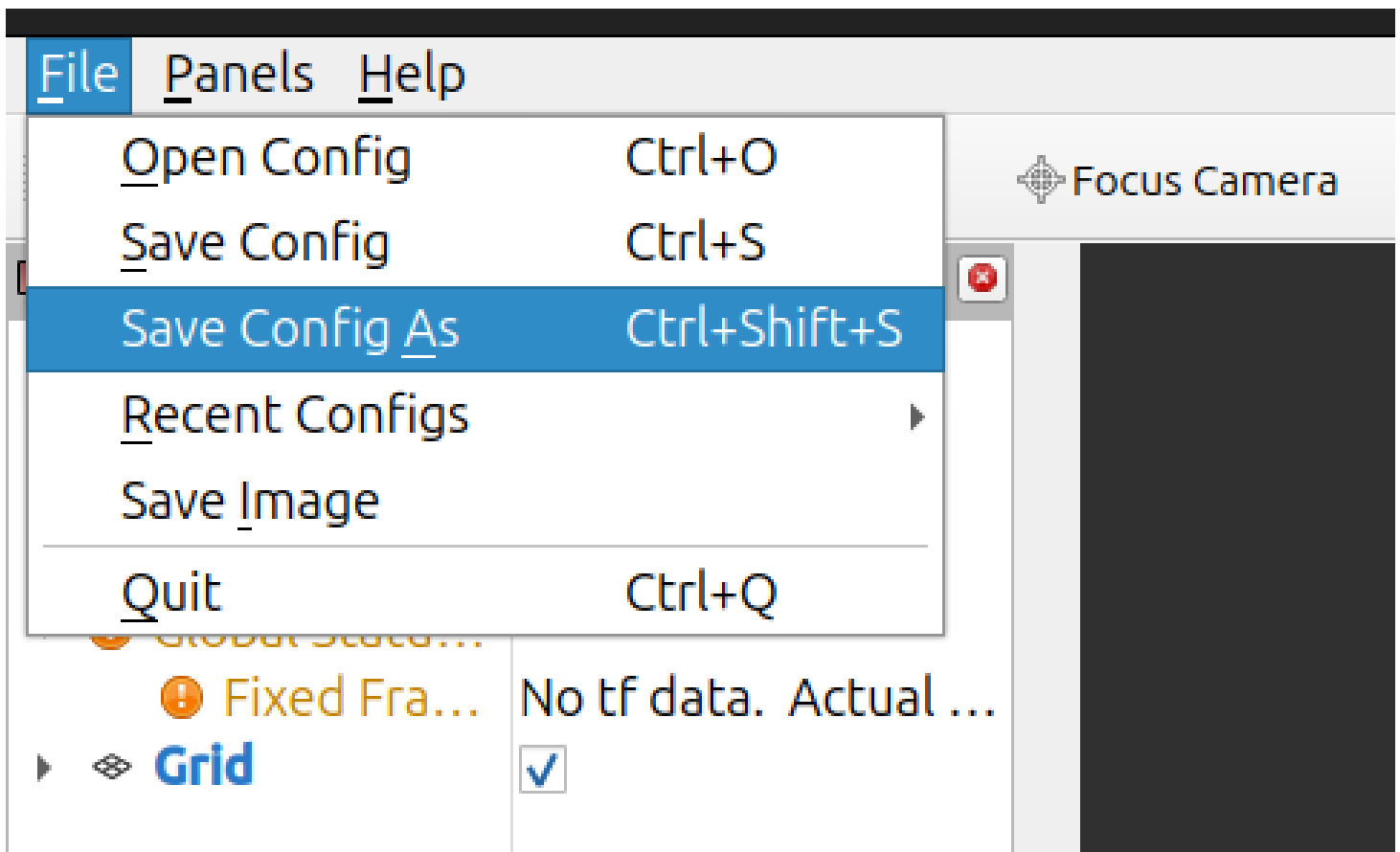


Figura 18: rviz2, file, save config as.

luego nombrar al archivo como panel.rviz, y guardar ese archivo en la carpeta `rviz` que acabamos de crear, como se muestra en la siguiente imagen:

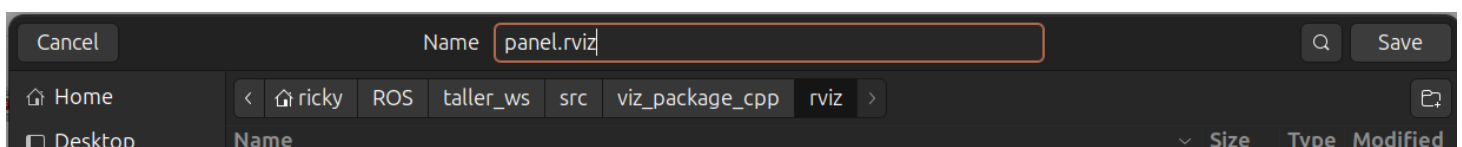
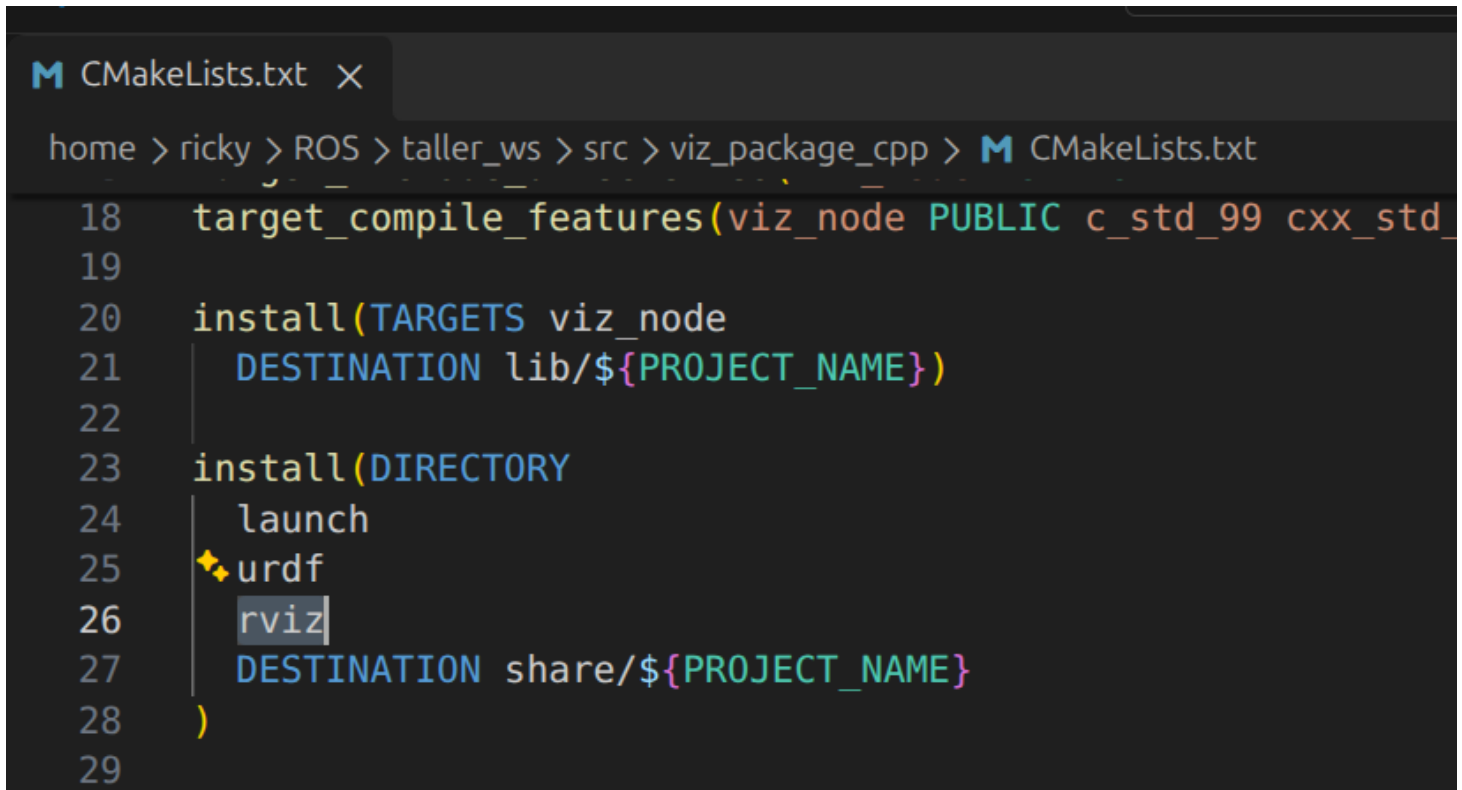


Figura 19: Guardamos archivo panel.rviz en la carpeta rviz.

Ahora vamos a agregar esta nueva carpeta `rviz` al install del archivo

CMakeLists.txt, como se muestra en la siguiente imagen.



```
18 target_compile_features(viz_node PUBLIC c_std_99 cxx_std_11)
19
20 install(TARGETS viz_node
21 DESTINATION lib/${PROJECT_NAME})
22
23 install(DIRECTORY
24 launch
25 urdf
26 rviz
27 DESTINATION share/${PROJECT_NAME})
28 )
29
```

Figura 20: Agregamos rviz al install del CMakeLists.txt

Luego, agregamos un nuevo nodo en el launch file del paquete, este nodo abrirá rviz2 cuando se ejecute. Modificamos el launch file como se ve en la siguiente imagen:

```
viz_launch.py x
home > ricky > ROS > taller_ws > src > viz_package_cpp > launch > viz_launch.py > generate_launch_description
7 def generate_launch_description():
20     output='screen',
21 ),
22 Node(
23     package='robot_state_publisher',
24     name='robot_state_publisher',
25     executable='robot_state_publisher',
26     output='screen',
27     parameters=[{
28         'robot_description': robot_desc,
29         'publish_frequency': 30.0,
30     }]
31 ),
32 Node(
33     package='rviz2',
34     name='rviz2',
35     executable='rviz2',
36     output='screen',
37     arguments=['-d', os.path.join(package_dir, 'rviz', 'panel.rviz')],
38 )
39 ])
```

Figura 21:

Ahora, en la terminal 1 compilamos con `colcon build`, en la terminal 2 lanzamos el launch file con `ros2 launch viz_package_cpp viz_launch.py`. Esto se muestra en las siguientes imagenes:

```
ricky@ThinkPad-T14-Gen-2i: ~/ROS/taller_ws 126x28
ricky@ThinkPad-T14-Gen-2i:~/ROS/taller_ws$ colcon build
Starting >>> viz_package_cpp
Finished <<< viz_package_cpp [0.73s]

Summary: 1 package finished [1.01s]
ricky@ThinkPad-T14-Gen-2i:~/ROS/taller_ws$ █
```

Figura 22:

```
ricky@ThinkPad-T14-Gen-2i:~/ROS/taller_ws$ ros2 launch viz_package_cpp viz_launch.py
[INFO] [launch]: All log files can be found below /home/ricky/.ros/log/2026-10-05-00-59-41-626532-ThinkPad-T14-Gen-2i-77184
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [viz_node-1]: process started with pid [77187]
[INFO] [viz_node-1]: process has finished cleanly [pid 77187]
[INFO] [robot_state_publisher-2]: process started with pid [77188]
[INFO] [rviz2-3]: process started with pid [77189]
[viz_node-1] hello world viz_package_cpp package
[robot_state_publisher-2] [INFO] [1791183581.736759256] [robot_state_publisher]: Robot initialized
[rviz2-3] [INFO] [1791183582.306080328] [rviz2]: Stereo is NOT SUPPORTED
[rviz2-3] [INFO] [1791183582.306377577] [rviz2]: OpenGL version: 4.6 (GLSL 4.6)
[rviz2-3] [INFO] [1791183582.420690192] [rviz2]: Stereo is NOT SUPPORTED
```

Figura 23:

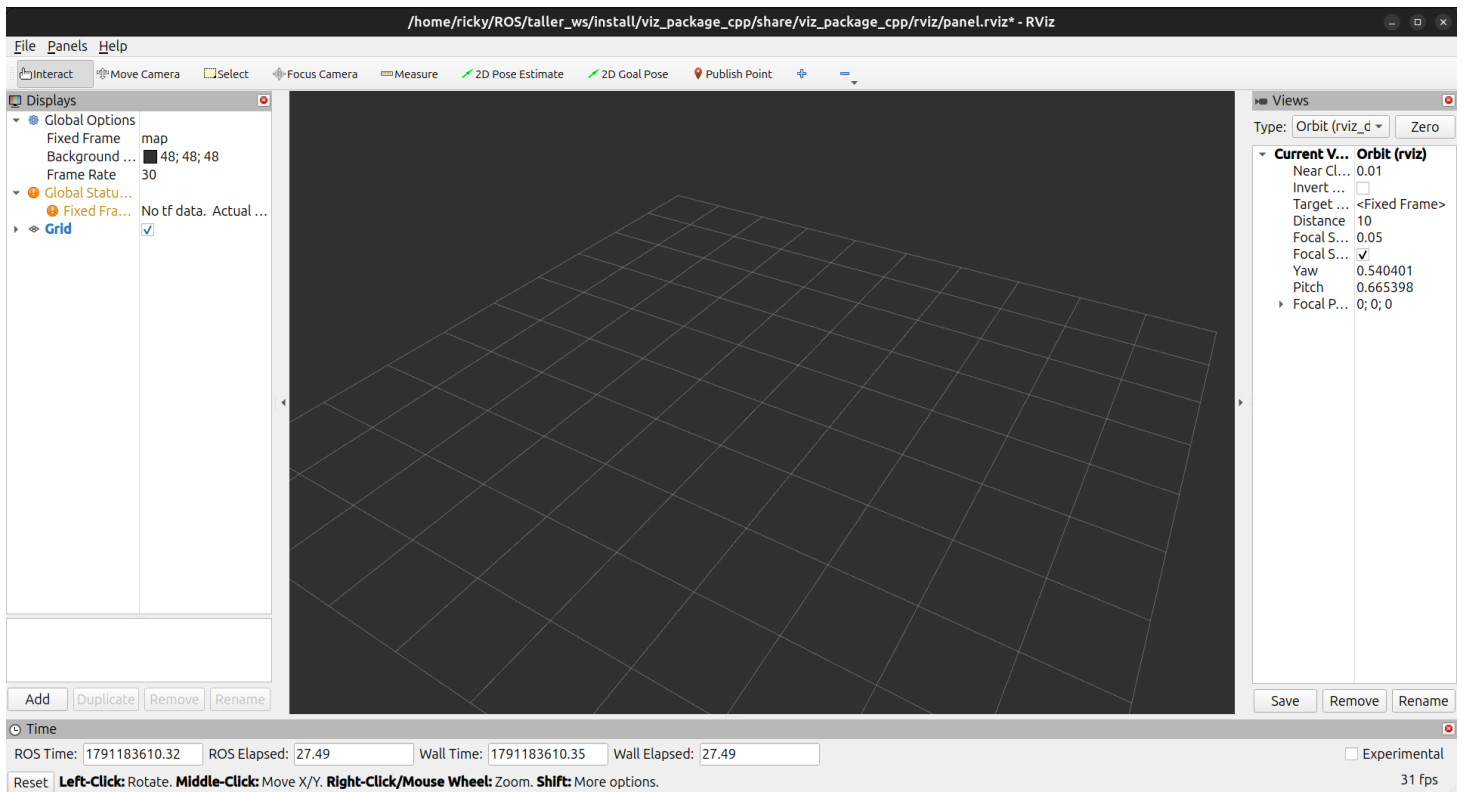


Figura 24:

Ahora, con el rviz abierto, en el menú de la izquierda, en la parte que dice Displays, seleccionamos el campo a la derecha de donde dice Fixed Frame, y cambiamos lo que dice map a base_link. Esto se muestra en las siguientes imágenes:

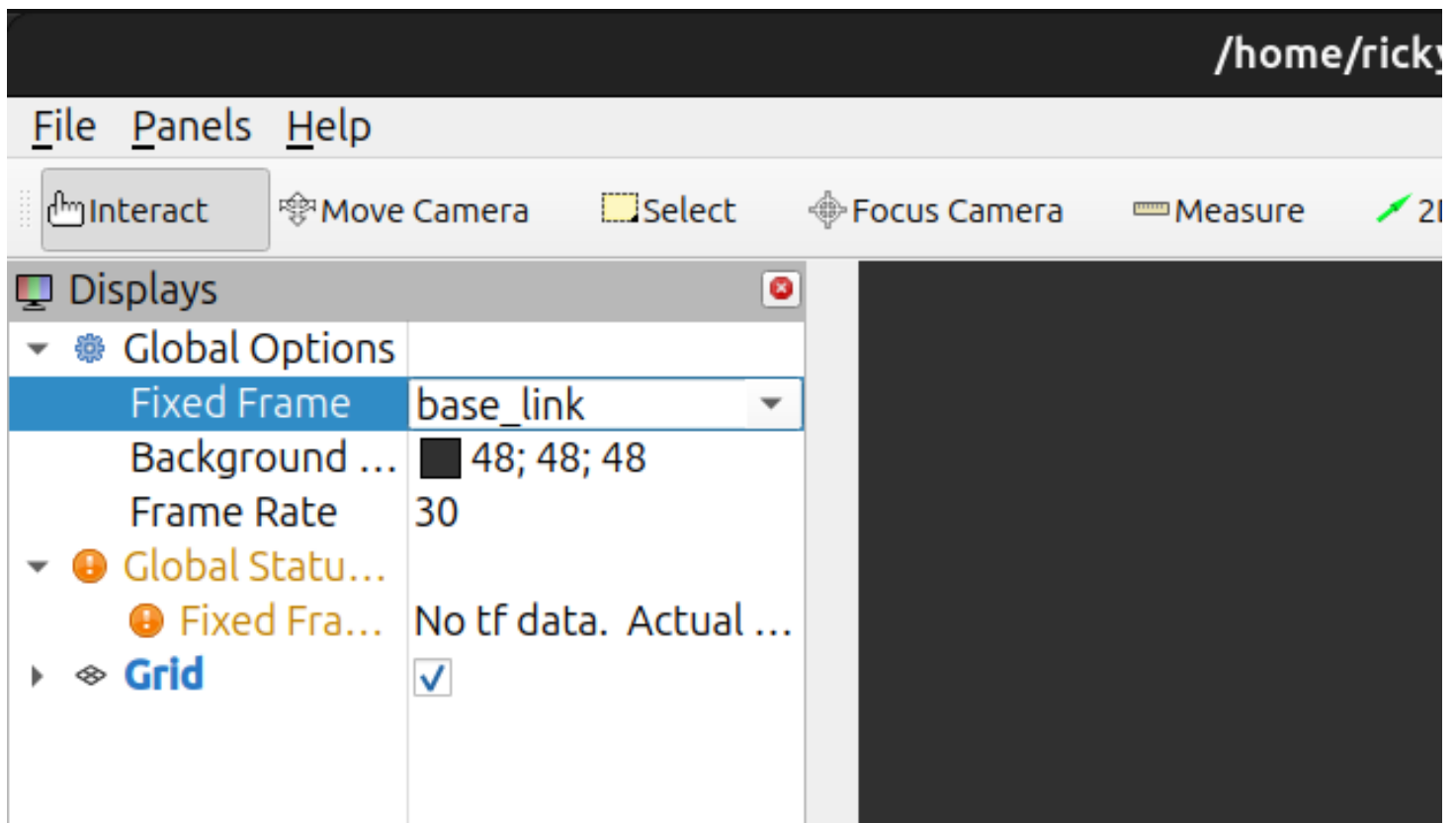


Figura 25:

Ahora, en rviz, seleccionamos el boton que dice Add (que está abajo a la izquierda), y en la pestaña que dice By Display Type, seleccionamos RobotModel. Como se ve en la siguiente imagen:

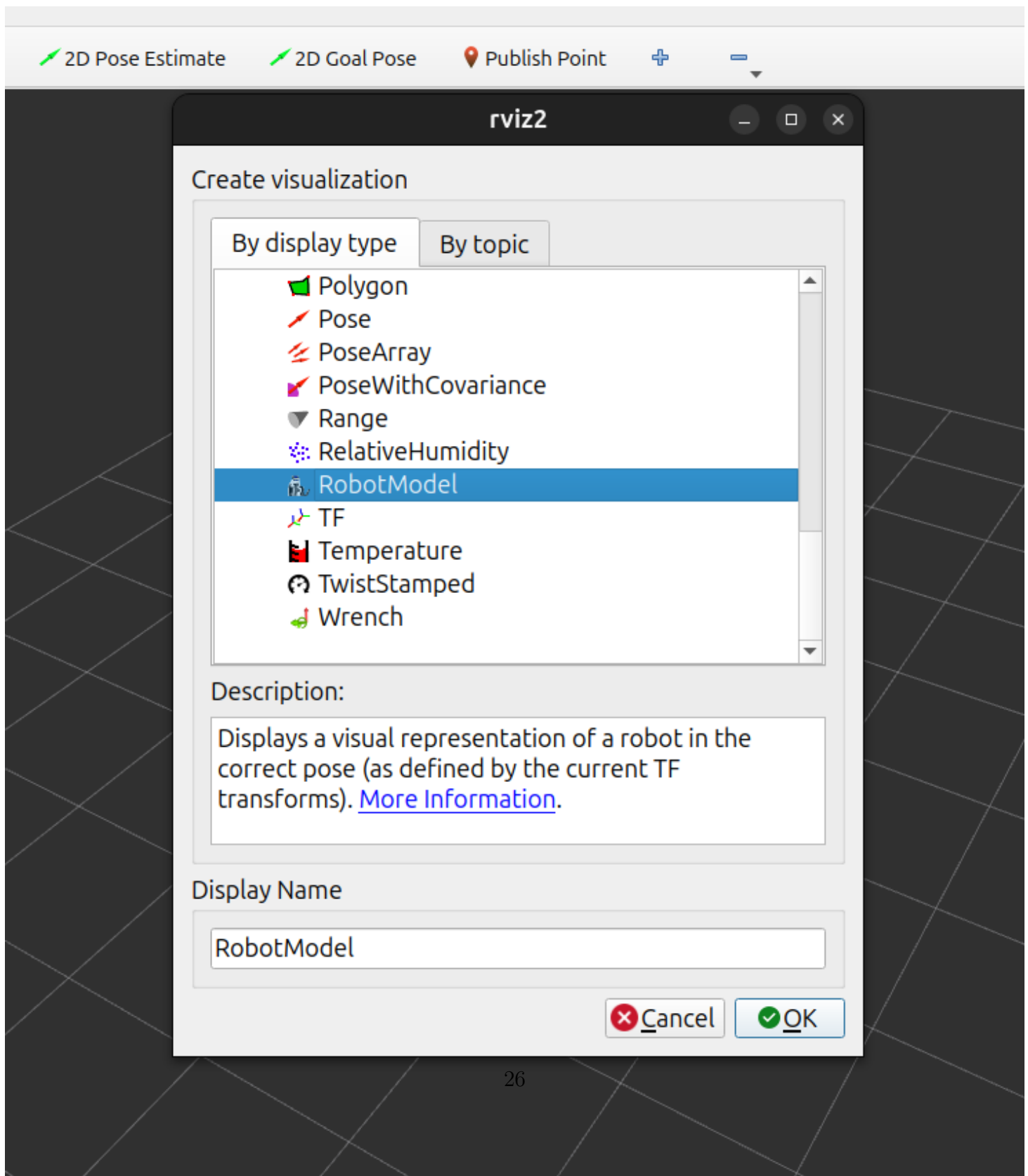


Figura 26:

Ahora, en el menu de la izquierda aparecerá en un nuevo apartado que dice RobotModel, y en la opción Description topic, seleccionamos la opción que dice /robot_description. Como se muestra en la imagen siguiente:

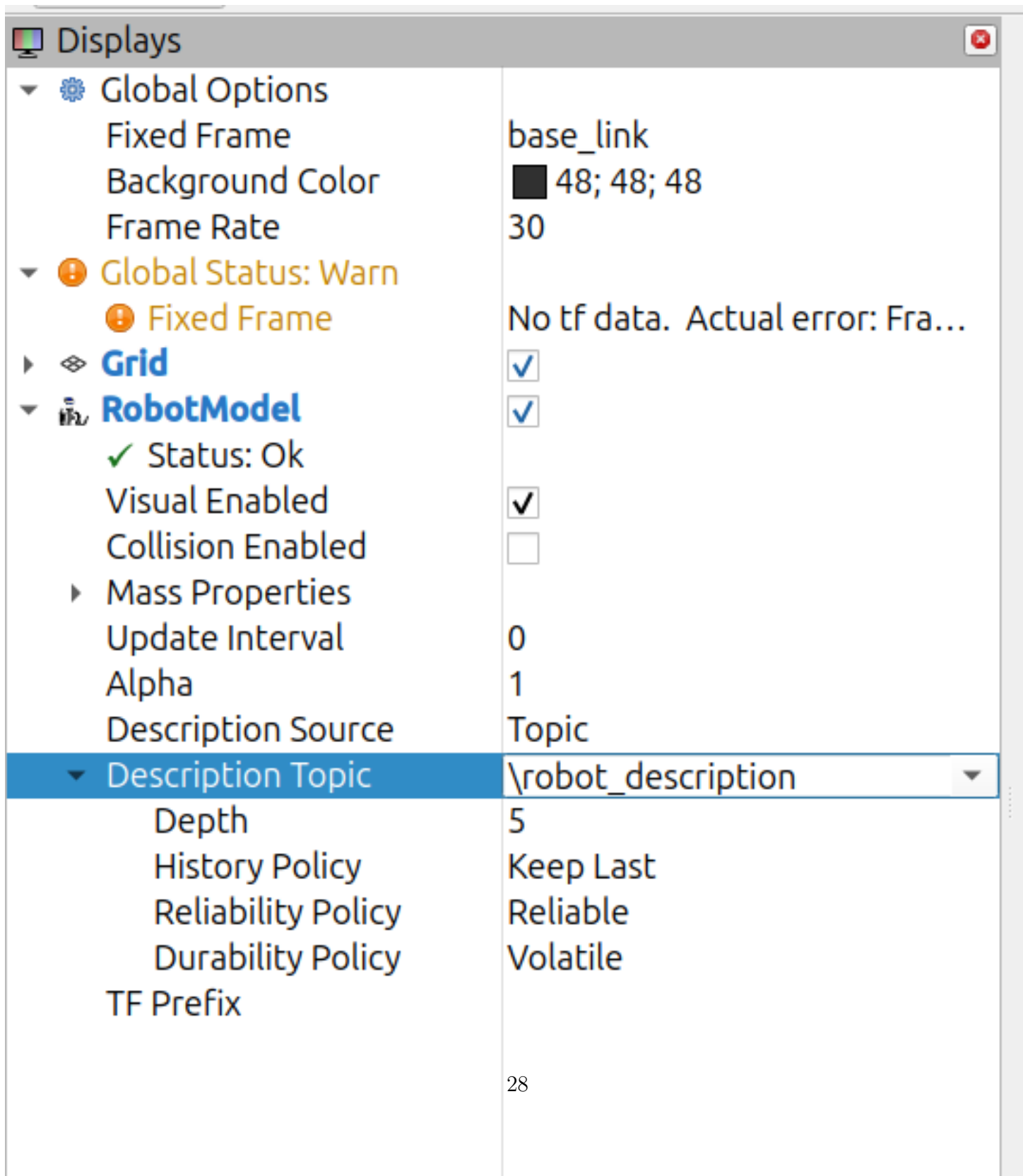


Figura 27:

Con las configuraciones anteriores, se debería de mostrar la pieza (nuestro robot hasta ahora) que definimos anteriormente en el archivo .urdf.

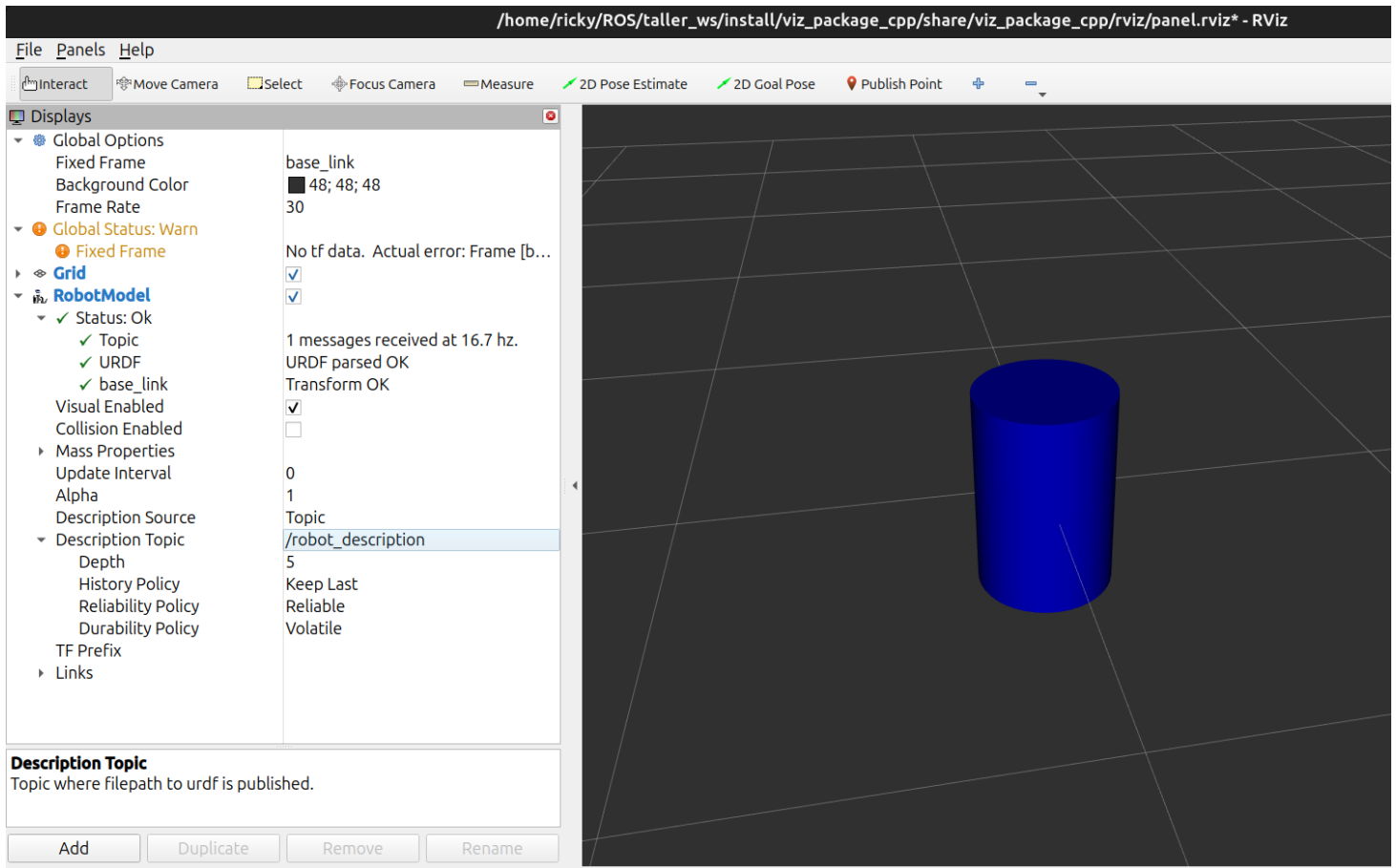


Figura 28: